

A Hierarchical Error-Corrective Graph Framework for Autonomous Agents with LLM-Based Action Generation

Cong Cao^a, Jingyao Zhang^b and Kun Tong^c

^aIndependent Researcher, Hangzhou, Zhejiang, China

^bBeihang University, Hangzhou, Zhejiang, China

^cBeihang University, Hangzhou, Zhejiang, China

ARTICLE INFO

Keywords:

LLM autonomous planning
robot action execution
graph retrieval
transition policy

ABSTRACT

While the integration of Reinforcement Learning (RL) and Large Language Models (LLMs) has enabled agents to generate high-level action plans for complex embodied tasks, robust execution remains an open challenge. Existing approaches struggle with three critical limitations: Firstly, traditional methods typically rely on single-dimensional metrics or simple weighted scoring mechanisms, which makes it difficult to comprehensively characterize the transferability of strategies in different tasks. This limitation is particularly pronounced in dynamic or partially observable environments. Secondly, current agent feedback mechanisms often focus solely on overall task success or failure, without providing structured attribution for the causes of failure. Finally, existing Retrieval-Augmented Generation (RAG) methods have achieved some success in mitigating LLM hallucinations, but their retrieval processes primarily depend on vector similarity or token-based matching, capturing only superficial semantic proximity and failing to fully leverage the structured relationships among historical experiences, actions, and events. This limitation restricts retrieval quality, semantic alignment, and contextual consistency. To address these issues, we propose a Hierarchical Error-Corrective Graph Framework (HECG) for Autonomous Agents with LLM-Based Action Generation, which incorporates three core innovations: (1) Multi-Dimensional Transferable Strategy (MDTS): By integrating task quality, execution cost, risk, and LLM-based semantic reasoning scores, MDTS achieves multi-dimensional alignment for the precise selection of high-quality candidate strategies, effectively reducing negative transfer risk. (2) Error Matrix Classification (EMC): Unlike simple confusion matrices, EMC provides structured attribution by categorizing errors into ten types based on severity and recoverability, offering clear and actionable guidance for subsequent error correction and replanning. (3) Causal-Context Graph Retrieval (CCGR): CCGR constructs graphs from historical states, actions, and event sequences to identify subgraphs most relevant to the current context. This captures deep structural dependencies beyond flat vector similarity, allowing agents to accelerate strategy adaptation and improve execution reliability. Extensive experiments in simulated embodied environments (e.g., VirtualHome) demonstrate that HECG significantly outperforms state-of-the-art LLM planners, achieving substantial improvements in task success rates, execution efficiency, and error recovery. Ultimately, our framework bridges the gap between high-level semantic reasoning and robust low-level execution in complex, multi-step tasks. The implementation and experimental code are publicly available at: <https://gitlab.com/magicmaster356a/cozy-hierplan>

1. Introduction

Recent advances in autonomous robotics and multi-robot systems have enabled increasingly complex tasks in unstructured and dynamic environments. Robots are now expected to perform sequential actions, adapt to unforeseen disturbances, and collaborate efficiently with other agents. Traditional approaches often include three core aspects of autonomous/multi-robot systems: Perception, Planning, and Collaboration [1].

A number of recent works have explored hybrid or learning-based solutions that partially alleviate execution uncertainty and coordination complexity. For instance, combining classical planning with reinforcement learning (RL) has shown promise in dynamic and multi-agent navigation settings. A recent study by Zhao [2] integrates Voronoi partitioning with deep RL to improve multi-robot exploration efficiency and dynamic obstacle avoidance in unknown environments. Similarly, decentralized, real-time, asynchronous

probabilistic trajectory planning frameworks have demonstrated improved success rates in cluttered and dynamically changing scenarios. In industrial and applied robotics, collaborative paradigms—such as those discussed in surveys from venues like Sukhatme’s research—emphasize robustness, safety, and real-world timing constraints, highlighting the need for adaptive and intelligent coordination strategies beyond idealized simulation environments [3]. However, while these approaches enhance adaptability and navigation robustness, they do not directly address several deeper structural limitations in transfer and error reasoning. Most existing transfer mechanisms still rely primarily on single-dimensional performance indicators (e.g., cumulative reward or success rate), which are insufficient to capture semantic compatibility and contextual alignment between source and target tasks. As a result, policy selection may remain vulnerable to negative transfer despite improved learning-based control. Likewise, execution failures are often summarized through aggregate metrics, without a structured mechanism to analyze failure type, source, and severity—limiting the system’s ability to perform systematic

*Corresponding author: magicmaster356a@gmail.com

ORCID(s):

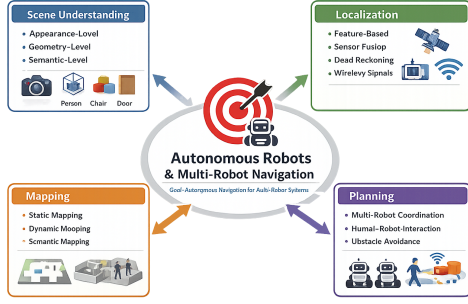


Figure 1: A categorization of autonomous robot methods.

corrective refinement. Furthermore, retrieval or reuse of prior experience in dynamic environments typically depends on flat similarity matching, overlooking the causal and sequential dependencies embedded in historical state-action trajectories, thereby constraining generalization and long-horizon adaptability.

This paper proposes a Hierarchical Error Correction (HEC) framework that integrates large language model based action generation with structured error-driven execution. In this framework, an LLM outputs a set of optional actions (action candidates), which are executed sequentially. Execution outcomes are monitored using task-specific error metrics:

- If the observed error is below a defined threshold, the system proceeds to the next step automatically.
- If the error exceeds the threshold, a multi-level correction mechanism is triggered.

The HECG framework operates across three hierarchical levels. At the first level, Local Correction performs fine-grained parameter adjustments to individual actions (e.g., minor end-effector repositioning). The second level, Optional Action Switching, enables strategy-level corrections by selecting alternative actions from a predefined optional set (e.g., switching from grasping to pushing when dealing with obstacles). Task Re-Planning is the third level, which provides high-level corrections by regenerating the entire action sequence while preserving historical failure information, ensuring that previously unsuccessful strategies are not repeated. A key innovation of this work is the integration of a graph-structured task representation with hierarchical error correction. Each node in the graph corresponds to an action or subgoal, encapsulating the action content, expected outcome, local threshold, and local correction rules. Edges represent task flows or optional transitions, including default paths, alternative branches, error-correction paths, and fallback mechanisms. This structure enables error-driven graph traversal, in which execution dynamically adapts to observed failures. When errors remain below predefined thresholds, the system continues along the main task path. If errors exceed these thresholds, corrective edges are activated to

invoke appropriate local or strategy-level corrections. In cases of persistent or repeated failures, higher-level fallback edges are triggered, potentially leading to task re-planning or escalation to human intervention.

By combining hierarchical correction with graph-based task representation, the proposed framework provides robust, adaptive, and interpretable decision-making for autonomous agents. The main contributions of this work are summarized as follows:

1. **Multi-Dimensional Transferable Strategy (MDTS):** We introduce a novel strategy selection mechanism that integrates task quality, execution cost, risk, and LLM-based semantic scores. This design enables precise strategy selection while effectively mitigating negative transfer across tasks.
2. **Error Matrix Classification (EMC):** We propose a structured error attribution mechanism that classifies task failures based on severity and recoverability. This framework provides actionable guidance for multi-level error correction and improves system robustness.
3. **Causal-Context Graph Retrieval (CCGR):** We design a graph-based retrieval method that captures both structural and causal dependencies in historical trajectories. Compared to traditional flat similarity matching, CCGR significantly enhances retrieval quality and long-horizon adaptability.

2. Related Work

2.1. LLM-based Planning for Embodied Agents

Recent advances in large language models (LLMs) have significantly influenced task planning and decision-making for embodied agents. By leveraging rich world knowledge and strong reasoning capabilities, LLMs have been used to generate high-level action sequences for robots in manipulation, navigation, and multi-step task execution scenarios. For example, representative systems integrate LLMs with feedback loops and modular recovery mechanisms to improve robustness in manipulation and navigation tasks, formalizing task planning as a goal-conditioned Markov decision process with explicit error mitigation strategies [4]. Other representative works such as SayCan [5], Joublin [6], IntelLiPlan [7] and ProgPrompt [8] demonstrate how LLMs can translate natural language instructions into executable robot actions by integrating symbolic reasoning with low-level controllers. Other efforts focus on constraining the output of an LLM for safer and more reliable planning, incorporating formal logic or chain-of-thought reasoning to reduce unsafe or infeasible action proposals in service robotics domains [9]. More recent research further explores structured alignment between high-level language reasoning and executable control primitives, introducing intermediate representations or action abstraction layers to ensure semantic consistency between generated plans and embodied affordances [10]. Such approaches enhance controllability and execution fidelity by bridging language-based reasoning with grounded policy execution. Additionally, hierarchical

LLM planners have been proposed to support multi-robot teams and event-driven replanning, demonstrating that structured decomposition and local event monitoring can enhance adaptability under execution disturbances [11]. These methods illustrate the ongoing push toward integrating symbolic reasoning and feedback awareness in LLM-based planners.

Despite these advances, many LLM planners still treat executable actions as fixed sequences with limited error-aware execution control. They often rely on coarse success/failure signals or post-hoc replanning without a systematic hierarchy of error thresholds and corrective actions during execution. In addition, these approaches typically assume reliable perception and actuation or rely on simple success/failure signals. As a result, LLM-generated plans often degrade in real-world environments due to sensor noise, execution uncertainty, or incomplete environment understanding. Several studies and surveys have reported that, without structured feedback or correction mechanisms, LLM-based planners often produce brittle plans that lack robustness under real-world disturbances. Works like Robot planning with LLMs highlight that language models alone are insufficient for responsive execution due to limited grounding in sensorimotor feedback, and must be integrated with perception and control modules to achieve reliable embodied behavior [12, 13, 14].

2.2. Error-aware Execution and Recovery

While LLM-based planners enhance high-level reasoning and task decomposition, robust embodied execution fundamentally depends on structured feedback and error-aware control mechanisms. Unlike the reasoning-oriented approaches discussed in Section 2.1, this line of work focuses on mitigating execution uncertainty arising from perception noise, actuation deviation, and environmental dynamics.

At the foundation of robotic robustness lies classical closed-loop control. Model-based controllers continuously incorporate sensor feedback to compensate for motion error and environmental perturbation [15]. These methods provide strong guarantees at the motion level, but they typically operate below the semantic task abstraction and do not explicitly reason about symbolic task failure. At the task level, structured execution frameworks such as Behavior Trees (BTs) and Finite State Machines (FSMs) introduce modular fallback and recovery policies [16]. By explicitly encoding failure conditions and recovery transitions, these representations improve predictability and reliability in industrial and service robotics. Their hierarchical control flow allows predefined corrective actions to be triggered under specific error states, enabling interpretable and safety-aware execution. However, their robustness largely depends on manually designed failure branches and domain-specific rules, limiting scalability to open-ended or novel tasks.

More recent research attempts to introduce adaptivity into structured planners. The SDA-PLANNER framework incorporates state dependency modeling and error-adaptive repair strategies, dynamically regenerating action subtrees

based on execution feedback [17]. Similarly, approaches combining LLM reasoning with semantic digital twins leverage environment-grounded simulation to iteratively refine plans and enable context-aware correction strategies, demonstrating improved robustness in embodied benchmarks such as ALFRED [18]. These efforts move beyond static fallback rules by incorporating environment-aware monitoring and partial replanning.

Learning-based methods further generalize recovery behaviors. Hierarchical reinforcement learning (HRL) decomposes complex tasks into sub-policies capable of local adaptation while maintaining global objectives [19]. Imitation learning has also been explored to capture human recovery strategies in manipulation and navigation tasks. These approaches show empirical gains in adaptability compared to purely rule-based systems.

Despite these advances, several limitations persist. First, many structured frameworks rely on reactive recovery triggered by binary success/failure signals, lacking graded or multi-level error modeling. Second, learning-based recovery mechanisms often require extensive training data and remain difficult to interpret or verify. Third, although recent systems attempt to integrate LLM reasoning with execution feedback, the coupling is frequently loose: high-level plans are corrected post hoc rather than being generated with explicit awareness of execution-level uncertainty. As a result, recovery remains local and reactive, rather than being embedded within a principled hierarchy of error thresholds and corrective strategies.

Overall, existing feedback-driven execution methods significantly improve robustness at the control and subtask levels. However, a systematic integration of high-level symbolic planning with structured, multi-level error management remains underexplored. This gap motivates the development of planning frameworks that incorporate error-awareness directly into the planning-execution loop.

2.3. Retrieval for Decision Making

Retrieval-Augmented Generation (RAG) has emerged as an effective paradigm for mitigating hallucinations and improving contextual grounding in Large Language Models. The original RAG framework proposed by Lewis et al. [20] integrates dense retrieval with sequence generation, enabling models to condition outputs on external knowledge sources. Subsequent work such as REALM by Guu et al. [21] further demonstrated the benefits of end-to-end differentiable retrieval for knowledge-intensive tasks. In embodied and decision-making settings, retrieval mechanisms have been increasingly adopted to provide contextual grounding for high-level planning and reasoning.

However, most existing RAG-based approaches rely primarily on vector similarity search, typically implemented using dense embeddings and nearest-neighbor retrieval. While effective for semantic matching, such methods often overlook structural, causal, and temporal dependencies embedded in embodied trajectories. In sequential decision-making

tasks, particularly under partially observed or dynamic environments, the relationships among states, actions, and events are inherently compositional and structured. Purely embedding-based retrieval may retrieve semantically similar but structurally incompatible trajectories, leading to suboptimal policy adaptation or inconsistent planning. To address these limitations, recent research has explored structured memory representations. Memory-augmented agents such as DeepMind’s Gato [22] and generative agents proposed by Park et al. [23] demonstrate how structured memory and event abstraction can improve long-horizon coherence and decision consistency. In robotics and embodied AI, approaches leveraging scene graphs, task graphs, and relational world models encode interactions among objects and actions as graph structures, enabling reasoning over affordances and causal dependencies rather than isolated tokens or embeddings [24].

Causal-Context Graph Retrieval (CCGR) further extend this idea by organizing historical trajectories into nodes (states, actions, subgoals) and edges (temporal transitions, causal effects, or semantic relations). Compared to flat vector databases, graph-structured memories preserve higher-order dependencies, enabling subgraph matching and structural similarity search [25, 26]. This allows agents to retrieve not only semantically related experiences but also structurally aligned execution patterns, which is particularly beneficial for multi-step task adaptation and transfer learning [27, 28]. Despite these advances, integration between retrieval and transfer evaluation remains limited. Existing methods often treat retrieval as a preprocessing step for generation, without jointly considering policy transferability, error profiles, or execution uncertainty. Moreover, vector-based retrieval alone cannot capture multi-dimensional transfer signals such as reward trends, confidence metrics, or LLM-derived semantic consistency scores [29]. In light of these limitations, our framework introduces Graph-Based Retrieval (Graph Retrieve) to structure historical states, actions, and event sequences into a graph memory. By performing subgraph-level relevance matching conditioned on current task context, the system captures structural dependencies beyond embedding similarity. Combined with our multi-dimensional transfer evaluation (Q, C, R, LLM-Score) and structured error matrix classification, this approach enables more reliable experience reuse, reduces negative transfer, and enhances execution robustness in complex embodied environments.

3. Method

3.1. Overview

Large Language Models (LLMs) have shown strong capabilities in high-level planning and sequential reasoning, enabling embodied agents to translate natural language instructions into action sequences. However, a key challenge remains: the mismatch between generated plans and environment dynamics often leads to execution failures, especially in long-horizon tasks.

This challenge arises from the *plan–environment alignment gap*, where the LLM’s implicit world model diverges from the grounded simulator state. Such misalignment can manifest as incorrect assumptions about object states, missing environmental preconditions, or stochastic execution outcomes. In long-horizon settings, these deviations accumulate over time, causing cascading failures and reducing overall task reliability.

To address this issue, we focus on *robust execution of LLM-generated plans* rather than improving plan generation itself. Given a natural language instruction T and an LLM-generated action sequence

$$\pi = (a_1, a_2, \dots, a_n), \quad (1)$$

our goal is to execute π reliably in stochastic and partially observable environments, minimizing failure propagation and unnecessary global replanning.

We propose the *Hierarchical Error-Corrective Control Graph (HECG)* framework, which reformulates plan execution as a *graph-based, feedback-driven process*. Specifically, we represent the plan as a directed graph

$$G = (V, E), \quad (2)$$

where nodes correspond to executable actions or subgoals, and edges encode transitions conditioned on execution outcomes and error types. This formulation enables dynamic adaptation during execution, rather than rigidly following a predefined sequence.

HECG integrates three key mechanisms. First, an *Error Matrix Classification (EMC)* module categorizes execution failures based on severity and recoverability, enabling structured error attribution. Second, a *Causal-Context Graph Retrieval (CCGR)* module retrieves relevant subgraphs from past trajectories, allowing the agent to reuse structured recovery strategies. Third, a *Multi-Dimensional Transfer Strategy (MDTS)* selects candidate actions or policies by jointly considering task quality, execution cost, risk, and semantic alignment. These components are unified through a *hierarchical transition policy* that organizes recovery into three levels: local action correction, alternative action selection, and task-level replanning. As a result, minor deviations are handled locally, while more severe inconsistencies trigger higher-level adaptations.

By combining structured error classification, graph-based retrieval, and hierarchical control, HECG transforms brittle sequential execution into a *robust, adaptive process*, improving long-horizon stability and reducing reliance on costly global replanning.

3.2. Graph-Based Retrieval

We store experience as a graph $G = (V, E)$ where nodes represent states, observations, actions, and outcomes, and edges encode temporal and causal relations (“action causes transition”, “state enables action”, “failure leads to recovery”). Given a current context, retrieval aims to identify a subgraph that matches both semantic intent (goal and

constraints) and structural similarity (relevant transition patterns). The retrieved subgraph provides (i) candidate actions, (ii) known failure modes, and (iii) recovery patterns to condition the LLM planner and the execution controller.

Each node $v_i \in V$ in the HECG represents an executable action or an intermediate subtask, serving as the fundamental unit for modeling both nominal execution and error-aware recovery. Formally, a node is defined as a tuple:

$$v_i = \langle t_i, a_i, \hat{o}_i, e_i, C_i, n_i \rangle \quad (3)$$

where each component explicitly captures different aspects of action execution under uncertainty. Specifically, t_i encodes task-level semantic information, including the subtask name and the set of relevant environment objects, grounding high-level task intent into a concrete execution context. The action a_i denotes the executable primitive or skill (e.g., *grasp*, *move*, *push*), which can be directly issued to the low-level controller or motion planner. The expected outcome $\hat{o}_i$ specifies the desired post-condition of the action, such as an object being grasped or a target location being reached, and serves as the reference for monitoring execution success.

The local error threshold e_i defines acceptable deviation bounds between the observed execution outcome and the expected outcome, enabling the system to distinguish between minor execution noise and significant failures. When deviations exceed this threshold, a set of predefined local correction rules C_i can be triggered to perform lightweight recovery actions, such as reattempting a grasp or adjusting the approach trajectory, without immediately invoking global replanning. Finally, n_i specifies the subsequent connected node(s), allowing the graph to represent both linear progressions and branching execution paths.

Transitions between nodes are encoded as directed edges $e_{ij} \in E$, which are defined as:

$$E \subseteq V \times K \times V, \quad e_{ij}^k = (v_i, k, v_j), \quad k \in K \quad (4)$$

Each edge captures not only the source and destination nodes but also the semantic type of the transition, explicitly modeling different execution and recovery behaviors. The edge type set is defined as:

$$K = \{\text{main}, \text{opt}, \text{corr}, \text{fb}\} \quad (5)$$

which includes:

- **Main execution edges:** define the nominal task flow generated by the LLM.
- **Optional edges:** connect alternative actions or skills that achieve the same subgoal, providing redundancy under execution uncertainty.
- **Correction edges:** activated when local errors exceed the predefined threshold, enabling targeted, node-level recovery.
- **Fallback edges:** allow the system to escalate from local correction to higher-level recovery strategies, such as switching subtask order or invoking task-level replanning.

By explicitly separating nominal execution, local correction, and high-level fallback mechanisms within the graph structure, HECG enables structured reasoning over failure modes and recovery strategies. This representation allows the agent to adapt its behavior dynamically based on real-time execution feedback, improving robustness while maintaining interpretability and reducing unnecessary global replanning.

3.3. Multi-Dimensional Transfer Strategy

3.3.1. Transition Policy

Given the HECG representation, where nodes encode executable actions or subtasks and edges encode nominal, optional, corrective, and fallback transitions, the remaining challenge is to determine which outgoing edge should be activated at each execution step under partial observability and execution uncertainty. Rather than relying on a fixed priority ordering or deterministic rules, we associate each candidate transition e_{ij}^k with a probabilistic transition policy that dynamically evaluates its suitability based on both structured signals and high-level semantic reasoning.

Specifically, at runtime, the agent maintains a belief state b_t capturing task progress and abstract execution context, as well as a low-level observation o_t reflecting the current environment and robot state. For each outgoing edge from the current node v_i , the agent computes a transition probability that integrates task value, execution cost, risk awareness, and LLM-based semantic feasibility. This policy formulation enables the HECG graph to function not merely as a static control structure, but as an adaptive decision graph that responds continuously to execution feedback. Formally, the probability of selecting a transition from node v_i to node v_j with semantic type k is defined as:

$$\pi_{ij}^k(b_t, o_t) = \text{Softmax} \left(\alpha Q_{ij}^k(b_t) - \beta C_{ij}(o_t) - \gamma R_{ij}(o_t) + \lambda \Phi_{ij}^{\text{LLM}}(b_t, o_t) \right) \quad (6)$$

This formulation decomposes transition selection into complementary components. The value term $Q_{ij}^k(b_t)$ captures long-horizon task utility and progress within the HECG graph, analogous to an MDP-style action-value function defined over abstract belief states. The execution cost $C_{ij}(o_t)$ penalizes transitions that are inefficient in terms of time, energy, or motion complexity, while the risk term $R_{ij}(o_t)$ estimates failure likelihood or safety hazards based on current observations. Crucially, the LLM-based score $\Phi_{ij}^{\text{LLM}}(b_t, o_t)$ injects semantic and commonsense reasoning into the decision process, allowing the agent to prefer transitions that are logically consistent with task intent, object affordances, and causal constraints that may not be explicitly encoded in low-level models. The coefficients $\alpha, \beta, \gamma, \lambda$ control the relative influence of these factors.

Each selected transition is grounded in a low-level skill $\kappa_{ij} \in \mathcal{K}_{\text{skill}}$, whose execution induces a stochastic observation transition, thereby closing the loop between symbolic decision-making and continuous control:

$$p(o_{t+1} \mid o_t, \kappa_{ij}) \quad (7)$$

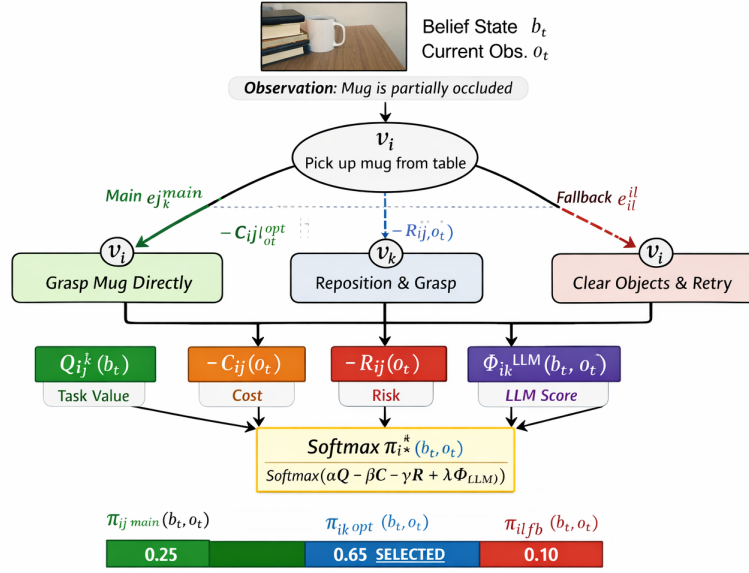


Figure 2: Structure of HECG Transition Policy. The agent selects among alternative action strategies by integrating task value, cost, risk, and LLM-based scores, and determines the final action via a softmax policy under partial observability.

Example: Transition Selection with LLM Participation. Consider a node v_i corresponding to the subtask “pick up a mug from the table”, with three outgoing transitions:

- Main edge e_{ij}^{main} : grasp mug directly.
- Optional edge e_{ik}^{opt} : reposition gripper and then grasp.
- Fallback edge e_{il}^{fb} : clear surrounding objects and retry.

Suppose the robot observes that the mug is partially occluded. The direct grasp transition has high task value Q but also elevated risk R . The optional transition incurs slightly higher cost C but lower risk. Meanwhile, the LLM assigns a high semantic feasibility score Φ^{LLM} to the optional transition, reasoning that “adjusting the gripper before grasping is appropriate when the object is occluded.” The fallback transition receives a lower LLM score, as clearing the table is semantically unnecessary at this stage. After combining all terms through the Softmax policy, the optional edge achieves the highest probability and is selected. If this transition later fails repeatedly, the accumulated risk increases and probability mass naturally shifts toward the fallback edge, enabling escalation without explicit rule-based switching. This example illustrates how HECG integrates structured decision signals and LLM reasoning to achieve adaptive and interpretable transition selection.

3.3.2. Control-Theoretic View of Error-Triggered Transitions

To explicitly model execution deviations and enable robust error handling, we define the local execution error for a node v_i as

$$e_i = \delta(o_t, \hat{o}_i) \quad (8)$$

where o_t is the current observation and $\hat{o}_i$ is the expected outcome of the action associated with v_i . This error metric quantifies the discrepancy between actual and desired outcomes, providing a principled basis for activating different transitions in the HECG. Using this error signal, transitions are triggered according to a state-dependent, threshold-based policy:

$$\pi_{ij}^k = \begin{cases} 1, & k = \text{main}, e_i \leq \epsilon_i \\ 2, & k = \text{corr}, \epsilon_i < e_i \leq \epsilon_i^{\text{max}} \\ 3, & k = \text{fb}, e_i > \epsilon_i^{\text{max}} \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

where ϵ_i and ϵ_i^{max} denote the local and maximum tolerable error thresholds, respectively. Intuitively, main edges are taken when execution is within acceptable bounds, correction edges are activated when the deviation exceeds local tolerance but remains recoverable, and fallback edges are triggered when errors surpass the maximum threshold, prompting higher-level replanning or recovery. This formulation naturally yields a hybrid switching system, where the HECG transitions are governed by state-dependent guards based on real-time execution feedback.

From a formal perspective, the HECG can be interpreted as a guarded finite-state automaton

$$A = (V, E, \Sigma, G) \quad (10)$$

where V and E correspond to the nodes and edges of the graph, Σ is the observation alphabet, and $G(e_{ij})$ denotes guard conditions defined over belief and error states. Each guard encodes the threshold logic described above, enabling the agent to switch between nominal, corrective, and fallback

behaviors in a structured and interpretable manner. This automaton view highlights the HECG's dual nature as both a planning graph and a control system, bridging symbolic task reasoning with control-theoretic robustness.

3.4. Error Matrix Classification

3.4.1. Three-Level Error Correction

We define an error matrix that indexes failures by *type* (e.g., perception, planning, control), *source* (sensor noise, occlusion, actuation slip, constraint violation), and *severity* (recoverable locally vs. requiring replanning). During execution, observed error signals are mapped into the matrix to decide the correction level:

- **L1 Local correction:** small continuous adjustments (re-grasp offsets, trajectory refinement).
- **L2 Option switching:** choose an alternative action or policy from a candidate set.
- **L3 Task replanning:** regenerate the plan with constraints derived from failure history.
- **L4 Human-in-the-loop:** safety or repeated failure triggers escalation.

The first level of correction focuses on local, low-cost recovery strategies that do not alter the global task structure. When the error slightly exceeds the predefined threshold, the system applies a set of local correction rules C_i associated with the current action node. These rules typically include continuous or parameter-level adjustments, such as fine-tuning the end-effector pose, reattempting a grasp with modified force or orientation parameters, or refining motion trajectories to compensate for minor disturbances. Since these corrections are computationally inexpensive and fast to execute, they enable rapid recovery from small, recoverable deviations while maintaining execution efficiency and stability.

If local corrections repeatedly fail or if the error magnitude exceeds a higher escalation threshold, the system transitions to optional action switching. At this level, the agent selects an alternative action node connected via optional edges in the Hierarchical Execution Control Graph (HECG). These optional actions represent discrete strategy variations that remain compatible with the overall task objective. For instance, if a direct grasp action fails persistently, the agent may switch to pushing the object into a more favorable configuration, repositioning the robot base, or approaching the object from a different direction. Optional actions can be generated dynamically by a Large Language Model (LLM) during planning or predefined based on domain expertise. This level enables structured strategy adaptation without triggering a full task replanning process.

When both local action correction and optional action switching fail to resolve the execution error, the system escalates to task-level replanning. At this stage, the complete execution context—including accumulated failure history, rejected action nodes, and updated environmental constraints—is fed back into the LLM to synthesize a revised

task plan. Crucially, actions that have previously failed are explicitly annotated to prevent their repetition in the newly generated plan. The updated plan is then re-encoded into a revised HECG, allowing execution to resume with improved robustness and informed decision-making. In extreme cases involving persistent failure or safety-critical conditions, the system may further escalate to a human-in-the-loop intervention node, ensuring safe and controlled recovery.

3.4.2. Error-Driven Graph Traversal Algorithm

Execution follows an error-driven traversal over the Hierarchical Error Correction Graph (HECG). The process begins by initializing the traversal at the root node, after which the action associated with the current node is executed in the environment. The system then evaluates the execution outcome by computing an error metric and comparing it against a predefined threshold. Based on the magnitude and type of the observed error, the hierarchical correction policy is invoked to select the most appropriate outgoing edge, determining whether to proceed, correct, or replan at a higher level of abstraction. The execution history is continuously updated to preserve contextual information for subsequent decisions, and this cycle repeats until the task is successfully completed or a termination condition is reached. By structuring execution as an adaptive traversal rather than a fixed sequence of actions, this mechanism enables the control flow to respond robustly to real-world uncertainties and execution failures.

3.4.3. Error Classification

To systematically handle execution failures, we categorize errors that may arise during task execution. Table 1 summarizes typical error types, severity levels, associated actions, and recoverability. Additional error categories are included beyond the initial four examples to cover more realistic scenarios in embodied execution.

This extended classification helps the HECG system distinguish between recoverable, partially recoverable, and unrecoverable failures, guiding the appropriate correction strategy—from local action adjustment to optional action switching, or task-level replanning.

4. Experiments

4.1. Tasks and Environments

All experiments are conducted in a simulated environment designed to evaluate hierarchical task planning and execution under realistic uncertainty. The widely-used embodied AI and task planning dataset *VirtualHome* provides programmatic household scenarios with annotated action sequences and includes diverse task types, ranging from simple single-object manipulations to complex multi-room interaction sequences. This makes it a comprehensive testbed for evaluating our proposed Hierarchical Error-Correcting Graph (HECG) framework.

In our experiments, we consider a set of representative tasks, including READBOOK, PUTDISHWASHER, PREPAREFOOD, and PUTFRIDGE, across different scenes such

Table 1
Error Classification in HECG Execution

Error Type	Severity	Typical Actions	Description	Recoverable	Correction Strategy	Whether Transition Needed
Action-Name-Mismatch-Error	High	[walk_to], [look_at]	Action name not found in supported actions.	Yes	<walk/walktowards>, <lookat>	Yes
Script-Parsing-Error	High	[putin] <obj>	Action lacks necessary parameters, resulting in a parsing failure.	No	[putin] <obj1> <obj2>	No
Action-Execution-Error	Medium	[open] <microwave>	Action failed because target object was not found, not reachable, or not visible.	Partially Yes	[switchon] <microwave>	Yes
Cascading-Execution-Failure	Low	[putin] <bananas> <fridge>	A cascading failure caused by previous unrecoverable action failures.	No	[open] <fridge> [putin] <bananas> <fridge>	Yes
Sensor-Failure-Error	Medium	–	Sensors failed to detect objects or environment state correctly.	Partially Yes	Reinitialize sensor pipeline; use redundant sensor data for fusion.	Yes
Collision-Detected-Error	High	[move], [push]	Robot collided with obstacle or object during execution.	Yes	Drop Action	Yes
Timeout-Error	Medium	–	Action did not complete within expected time limits.	Partially Yes	Retry Action	Yes
Hardware-Fault-Error	Critical	–	Physical actuator or gripper malfunction prevents action execution.	No	Emergency stop; notify human operator.	No
Perception-Mismatch-Error	Medium	[open] <fridge> (fridge already opened)	Perceived object pose differs from expected pose; causes partial failure.	Yes	[close] <fridge> [open] <fridge>	Yes
Agent-Positioning-Error	Medium	[lookat] <kitchentable>	Agent is not correctly localized relative to target object or navigation point, leading to approach failure.	Yes	<walk> kitchen, <lookat> <kitchentable>	Yes

as BEDROOM, LIVINGROOM, KITCHEN, and BATHROOM. These tasks require multi-step reasoning, sequential dependencies, and interaction with dynamic objects. Each task is represented as a hierarchy of nodes, where each node corresponds to either a primitive action or a higher-level subtask. During execution, observations are generated at each step and compared against expected outcomes to detect potential errors, following the formulation described in Section 3.

We implement the proposed Hierarchical Error-Correcting Graph (HECG) agent using a modular architecture that integrates three core components: planning, verification, and replanning. For the LLM-based baseline, we employ a large language model (LLM) to directly generate a sequence of actions given the task description, without incorporating explicit hierarchical error correction or transition policies.

We compare the HECG framework against the following baselines:

- **LLM Planner (Flat):** A conventional LLM-based planner that generates the entire action sequence in a

single pass, without hierarchical correction or replanning mechanisms.

- **HECG w/o Transition:** A variant of our model that includes hierarchical error correction but removes the learned transition policy, allowing us to measure the impact of explicit state transitions between subtasks.
- **HECG Full:** The complete HECG agent with both hierarchical correction and transition policies enabled.

4.2. Baselines

All experiments are conducted in a simulated environment designed to evaluate hierarchical task planning and execution under realistic uncertainty. The widely-used embodied AI and task planning dataset *VirtualHome* provides programmatic household scenarios with annotated action sequences and includes diverse task types, ranging from simple single-object manipulations to complex multi-room interaction sequences. This makes it a comprehensive testbed for evaluating our proposed Hierarchical Error-Correcting Graph (HECG) framework.

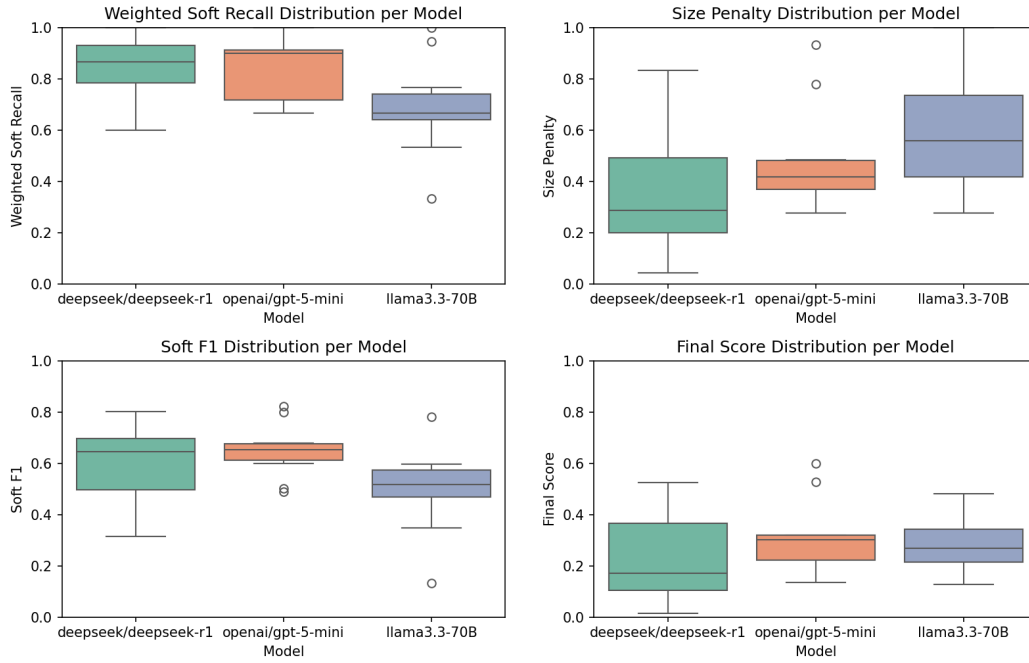


Figure 3: LLM Goal Compliance Evaluation Results.

In our experiments, we consider a set of representative tasks, including READBOOK, PUTDISHWASHER, PREPAREFOOD, and PUTFRIDGE, across different scenes such as BEDROOM, LIVINGROOM, KITCHEN, and BATHROOM. These tasks require multi-step reasoning, sequential dependencies, and interaction with dynamic objects. Each task is represented as a hierarchy of nodes, where each node corresponds to either a primitive action or a higher-level subtask. During execution, observations are generated at each step and compared against expected outcomes to detect potential errors, following the formulation described in Section 3.

We implement the proposed Hierarchical Error-Correcting Graph (HECG) agent using a modular architecture that integrates three core components: planning, verification, and replanning. For the LLM-based baseline, we employ a large language model (LLM) to directly generate a sequence of actions given the task description, without incorporating explicit hierarchical error correction or transition policies.

We compare the HECG framework against the following baselines:

- **LLM Planner (Flat):** A conventional LLM-based planner that generates the entire action sequence in a single pass, without hierarchical correction or replanning mechanisms.
- **HECG w/o Transition:** A variant of our model that includes hierarchical error correction but removes the learned transition policy, allowing us to measure the impact of explicit state transitions between subtasks.
- **HECG Full:** The complete HECG agent with both hierarchical correction and transition policies enabled.

4.3. Overall Performance Comparison (Goal Compliance & Task Plan)

Goal Compliance Evaluation. We first evaluate *goal compliance*, which measures the extent to which the final world state satisfies the intended objectives of a task. Unlike task success rate, which requires all actions to be executed correctly, goal compliance focuses on whether the desired end state is achieved, even if intermediate steps are suboptimal. Formally, for a task T with a set of goal conditions

$$G = \{g_1, g_2, \dots, g_n\}, \quad (11)$$

goal compliance is defined as:

$$\text{Goal Compliance} = \frac{|\{g \in G \mid g \text{ satisfied in final state}\}|}{|G|} \quad (12)$$

where $|G|$ denotes the total number of goal conditions, and the numerator counts the number of satisfied conditions after execution.

We evaluated three models (GPT-5 Mini, DeepSeek-R1, and LLaMA3.3-70B) on multi-room household task execution scenarios. The tasks involve cross-room object interactions, including read book, put dishwasher, prepare food, put fridge, and setup table. Evaluation metrics include weighted soft recall, soft precision, soft F1, size penalty, and a final composite score reflecting overall execution quality adjusted for sequence length.

Across all evaluated tasks, GPT-5 Mini demonstrates the strongest overall performance, achieving the highest average final score (0.315), compared with LLaMA3.3-70B (0.278) and DeepSeek-R1 (0.230). GPT-5 Mini maintains a

Hierarchical Error-Corrective Graph Framework

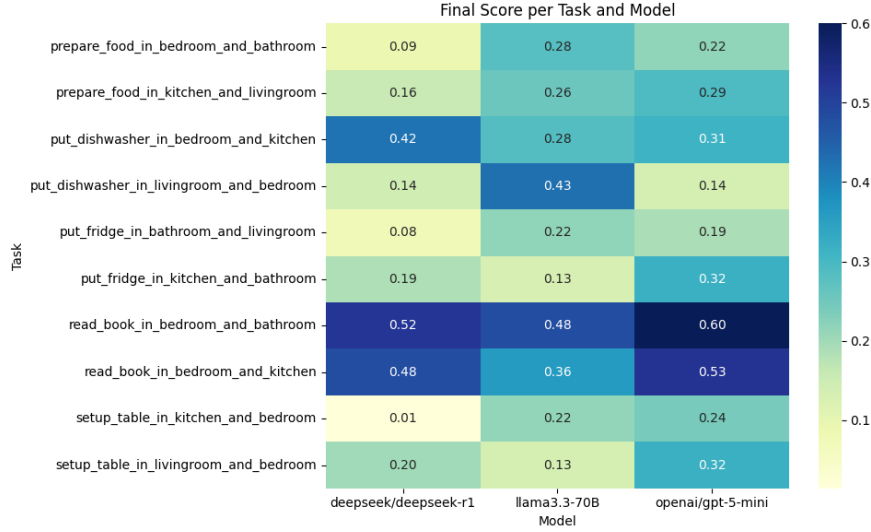


Figure 4: Heatmap of Goal Compliance.

strong balance between recall and precision, with an average weighted soft recall of 0.842 and soft precision of 0.534, resulting in the highest average soft F1 (0.651) among the evaluated models. In particular, GPT-5 Mini performs best on put fridge in kitchen and bathroom, achieving an F1 score of 0.823 and a final score of 0.320, and also on setup table in livingroom and bedroom, where it reaches perfect recall (1.0) with an F1 score of 0.800 and a final score of 0.320. However, tasks such as put dishwasher in livingroom and bedroom yield lower scores (final score 0.135), likely due to increased spatial complexity and multi-step dependencies, which reduce precision and incur stronger sequence length penalties.

DeepSeek-R1 achieves the highest average recall (0.852) among the models and performs strongly on tasks such as put dishwasher in bedroom and kitchen (final score 0.422). However, it frequently generates longer action sequences, resulting in larger size penalties, which significantly reduce its overall composite score.

LLaMA3.3-70B demonstrates relatively balanced behavior but tends to produce longer plans with lower precision (average 0.400), which limits its final performance. While it performs competitively on tasks such as put dishwasher in livingroom and bedroom (final score 0.427), it also exhibits unstable behavior on some tasks (e.g., setup table in livingroom and bedroom, final score 0.133).

Overall, GPT-5 Mini achieves the best balance between recall, precision, and sequence efficiency, indicating stronger grounding and structured sequence planning capability. DeepSeek-R1 tends to prioritize recall at the expense of efficiency, while LLaMA3.3-70B exhibits moderate recall but reduced precision and longer action sequences.

Task Success and Action-Level Evaluation. We further conduct a comparative evaluation of basic household task execution across paired scenes (e.g., bedroom_and_kitchen,

kitchen_and_bathroom). The evaluation metrics include Task Success Rate (%), Average Action Accuracy (%), and Total Steps.

Success Rate (SR). Success rate measures the proportion of episodes that achieve the task goal. We distinguish between two types:

- **Original Success Rate:** The success rate of executing the initially generated plan without any replanning.
- **Replan Success Rate:** The success rate achieved after incorporating replanning and error recovery mechanisms.

The final success rate for a task is defined as:

$$SR_{\text{final}} = \frac{N_{\text{successful episodes}}}{N_{\text{total episodes}}} \quad (13)$$

Action Accuracy (AA) Action Accuracy measures the correctness of individual actions executed by the agent in a task episode. It is defined as the ratio of correctly executed actions to the total number of actions taken:

$$AA = \frac{N_{\text{correct actions}}}{N_{\text{executed actions}}} \quad (14)$$

where:

- $N_{\text{correct actions}}$ is the number of actions that match the reference or ground-truth action sequence.
- $N_{\text{executed actions}}$ is the total number of actions executed by the agent in the episode.

A value of 1 indicates that all actions in the episode were executed correctly, while lower values reflect incorrect or mismatched actions. Action Accuracy provides a fine-grained, step-level measure of procedural correctness, complementing task-level success metrics.

Efficiency Score. Efficiency quantifies how optimally the plan achieves the task goal, measured by the number of executed actions relative to the optimal plan length:

$$\text{Efficiency} = \frac{N_{\text{optimal steps}}}{N_{\text{executed steps}}} \quad (15)$$

A value of 1 indicates perfect efficiency matching the optimal plan, while lower values indicate additional steps due to errors or inefficient recovery actions.

Stability Analysis. We assess the consistency of model performance using the coefficient of variation (CV) of success rates across episodes:

$$\text{CV} = \frac{\sigma_{\text{SR}}}{\mu_{\text{SR}}} \quad (16)$$

where σ_{SR} is the standard deviation and μ_{SR} is the mean of success rates across episodes. Lower CV values indicate more stable and reliable performance.

Task Complexity Assessment. To analyze the relationship between task difficulty and model performance, we compute an overall complexity score for each task based on:

- Number of required actions
- Environmental constraints and object interactions
- Sequential dependencies between subgoals
- Potential error-prone steps

This complexity measure enables analysis of how different models scale with task difficulty. Task complexity significantly affects step count. For instance, *Preparefood* tasks require 15–19 steps, reflecting increased dependency chains. LLaMA3.3-70B generally uses fewer steps than DeepSeek-R1 for comparable tasks, suggesting relatively more efficient action sequencing.

Improvement Metrics. We quantify the benefit of replanning through improvement in success rate:

$$\text{Improvement} = \text{SR}_{\text{replan}} - \text{SR}_{\text{original}} \quad (17)$$

Positive values indicate that replanning effectively recovers from execution errors.

Model Performance Analysis We conducted a comparative evaluation of household task execution across paired scenes (Displayed in Table 2 and Figure 4). The evaluation metrics include **Original Success Rate (SR, %)**, **Average Action Accuracy (AA, %)**, and **Total Steps**.

Model Performance. GPT-5 Mini demonstrates consistently high **action-level accuracy**, particularly in *Preparefood* tasks, occasionally reaching 0.968. This indicates strong procedural correctness even when full task completion is not achieved. DeepSeek-R1 shows moderate but variable performance: it can achieve high action accuracy in some tasks, such as 0.933 for *Readbook* in *bedroom_and_bathroom*, although high action accuracy does

Table 2

Original Task Performance Metrics Across Models

Task (Scene)	Model	Original Success Rate (%)	Average Action Accuracy (%)	Total Steps
Readbook (bedroom&kitchen)	Llama3.3-70B	0.797	0.833	7
	Deepseek-R1	0.854	0.725	8
	GPT-5-mini	0.767	0.643	5
Readbook (bedroom&bathroom)	Llama3.3-70B	0.765	0.677	6
	Deepseek-R1	0.831	0.933	8
	GPT-5-mini	0.722	0.744	7
Putdishwasher (bedroom&kitchen)	Llama3.3-70B	0.711	0.866	9
	Deepseek-R1	0.792	0.867	13
	GPT-5-mini	0.645	0.755	10
Putdishwasher (livingroom&bedroom)	Llama3.3-70B	0.761	0.833	12
	Deepseek-R1	0.828	0.713	11
	GPT-5-mini	0.715	0.856	10
Preparefood (kitchen&livingroom)	Llama3.3-70B	0.734	0.833	19
	Deepseek-R1	0.808	0.865	15
	GPT-5-mini	0.677	0.955	17
Preparefood (bedroom&bathroom)	Llama3.3-70B	0.734	0.833	18
	Deepseek-R1	0.808	0.725	16
	GPT-5-mini	0.677	0.968	17
Putfridge (bathroom&livingroom)	Llama3.3-70B	0.749	0.885	19
	Deepseek-R1	0.819	0.756	15
	GPT-5-mini	0.699	0.643	15
Putfridge (kitchen&bathroom)	Llama3.3-70B	0.730	0.856	18
	Deepseek-R1	0.806	0.725	14
	GPT-5-mini	0.671	0.855	17
Setuptable (kitchen&bedroom)	Llama3.3-70B	0.748	0.844	17
	Deepseek-R1	0.819	0.885	15
	GPT-5-mini	0.698	0.889	16
Setuptable (livingroom&bedroom)	Llama3.3-70B	0.760	0.755	14
	Deepseek-R1	0.827	0.725	16
	GPT-5-mini	0.714	0.855	17

not always correspond to higher task success. LLaMA3.3-70B exhibits robust **scene-level grounding**, often achieving slightly higher success rates in spatially complex tasks while generally completing tasks with fewer steps than DeepSeek-R1, suggesting more efficient action sequencing.

Task Complexity and Efficiency. Task complexity significantly affects the number of steps required. *Preparefood* tasks, for example, require 15–19 steps due to longer dependency chains and multiple object interactions. **Efficiency**, defined as the ratio of optimal plan length to executed steps, reflects how closely models follow the optimal sequence; values closer to 1 indicate efficient execution, while lower values reflect additional steps caused by errors or recovery actions.

Stability and Error Analysis. We assess performance consistency using the **coefficient of variation (CV)** of success rates across episodes. Lower CV indicates more stable and reliable performance. Execution errors are quantified via **Average Error Count (EC)**, which measures the mean number of errors per episode, and **Fatal Error Rate (FER)**, representing the proportion of episodes with unrecoverable errors leading to task failure.

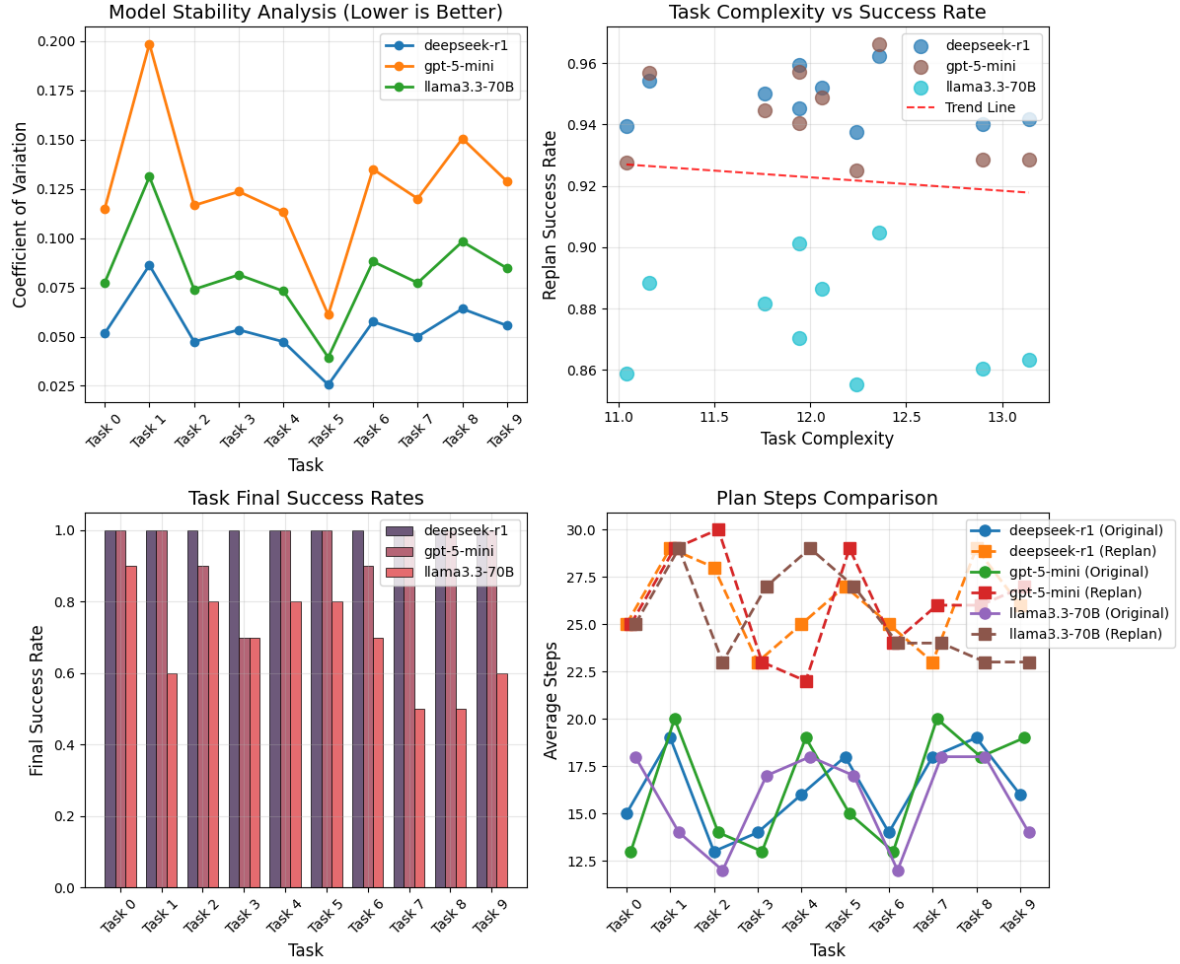


Figure 5: Evaluation Metrics of Task Plan

Improvement through Replanning. Replanning mechanisms can improve task success by recovering from execution errors. **Improvement** is measured as the difference between success rates after replanning and the original plan, with positive values indicating effective error recovery.

In summary, all models show baseline competency in household task execution, but with different strengths. LLaMA3.3-70B excels in **scene-level robustness**, GPT-5 Mini in **action-level precision**, and DeepSeek-R1 in **recall**, though with variable planning efficiency. Task complexity and sequential dependencies have a notable impact on both step count and overall task success, highlighting the trade-offs between efficiency, accuracy, and robustness across models.

4.4. Ablation on Hierarchical Correction Levels

We evaluated the performance of three models—LLaMA3.3-70B, Deepseek-R1, and GPT-5-mini—on multiple Virtual-Home tasks across different room settings. The evaluation metrics include:

1. **Task Success Rate (TSR):** The overall measures the average goal completion ratio across multiple task

executions. For each execution, TSR is computed as the ratio of successfully achieved goals to the total number of goals, and the final TSR is obtained by averaging this ratio across all executions. Formally defined as follows:

$$\text{TSR} = \frac{1}{N} \sum_{i=1}^N \frac{N_{\text{success}}}{N_{\text{total}}} \quad (18)$$

where N_{success} is the number of goals completed successfully and N_{total} is the total number of expected goals, and N denotes the total trial numbers of each task.

2. **TSR_R (Replan Success Rate):** TSR_R measures the effectiveness of the replanning mechanism in recovering from execution failures. Specifically, it evaluates the proportion of goals that are successfully completed after at least one execution failure followed by a replanning procedure. Only task executions that experience at least one execution failure are included in the computation of TSR_R; executions without

failures contribute zero to the metric:

$$TSR_R = \sum_{i=1}^N \frac{N_{\text{success with replan}}^i}{N_{\text{total}}^i} \quad (19)$$

where $N_{\text{success after replan}}^i$ denotes the number of goals that are eventually achieved through replanning following an execution failure, and N_{total}^i is the total number of goals in task i .

3. **TSR_C (Corrective Success Rate):** TSR_C evaluates the effectiveness of corrective actions applied during execution to recover from failed steps after replanning. It measures the proportion of replanning-enabled successes that are ultimately achieved through explicit corrective actions. The overall TSR_C across N task executions is computed as:

$$TSR_C = \sum_{i=1}^N \frac{N_{\text{success with correction}}^i}{N_{\text{success with replan}}^i} \quad (20)$$

where $N_{\text{success with correction}}^i$ denotes the number of goals that are successfully completed through corrective actions, and $N_{\text{success after replan}}^i$ is the number of goals that remain achievable after replanning.

4. **Error Rate (ER):** Proportion of tasks that encountered unrecoverable failures. Execution errors are categorized into four types:
 - *Grounding Errors:* failure to identify or locate an object in the environment.
 - *Precondition Errors:* attempting actions when necessary preconditions are not met.
 - *Affordance Errors:* attempting impossible interactions with objects.
 - *Execution Errors:* failures due to environment dynamics or script infeasibility.

For each type, we calculate the proportion relative to all errors:

$$\text{Error Ratio}_{\text{type}} = \frac{N_{\text{errors of type}}}{N_{\text{total errors}}} \quad (21)$$

From the results, several trends can be observed across tasks and models. To begin with, Deepseek-R1 generally achieves the highest original task success rate (TSR) across most task–scene combinations. For instance, in tasks such as Readbook and Preparefood, Deepseek-R1 consistently outperforms the other models in TSR, suggesting stronger initial planning ability before any corrective mechanisms are applied. However, once corrective mechanisms are introduced, a different pattern emerges. GPT-5-mini demonstrates the strongest performance after corrective actions, with TSR_C values close to or equal to 1.0 in most tasks. This indicates that GPT-5-mini is particularly effective at utilizing corrective feedback to recover from execution failures. Even when its initial TSR is slightly lower than that of Deepseek-R1, the model is often able to successfully complete the task

Table 3

Performance comparison across different models and tasks.

Task (Scene)	Model	TSR	TSRR	TSRC	ER
Readbook (bedroom&kitchen)	Llama3.3-70B	0.797	0.888	0.900	0.677
	Deepseek-R1	0.854	0.954	1.000	0.717
	GPT-5-mini	0.767	0.957	1.000	0.797
Readbook (bedroom&bathroom)	Llama3.3-70B	0.765	0.886	0.600	0.504
	Deepseek-R1	0.831	0.952	1.000	0.128
	GPT-5-mini	0.722	0.949	1.000	0.752
Putdishwasher (bedroom&kitchen)	Llama3.3-70B	0.711	0.882	0.800	0.317
	Deepseek-R1	0.792	0.950	1.000	0.794
	GPT-5-mini	0.645	0.945	0.900	0.595
Putdishwasher (livingroom&bedroom)	Llama3.3-70B	0.761	0.860	0.700	0.833
	Deepseek-R1	0.828	0.940	1.000	0.992
	GPT-5-mini	0.715	0.928	0.700	0.595
Preparefood (kitchen&livingroom)	Llama3.3-70B	0.734	0.905	0.800	0.474
	Deepseek-R1	0.808	0.962	1.000	0.839
	GPT-5-mini	0.677	0.966	1.000	0.730
Preparefood (bedroom&bathroom)	Llama3.3-70B	0.734	0.901	0.800	0.682
	Deepseek-R1	0.808	0.959	1.000	0.230
	GPT-5-mini	0.677	0.957	1.000	0.644
Putfridge (bathroom&livingroom)	Llama3.3-70B	0.749	0.859	0.700	0.962
	Deepseek-R1	0.819	0.939	1.000	0.808
	GPT-5-mini	0.699	0.928	0.900	0.423
Putfridge (kitchen&bathroom)	Llama3.3-70B	0.730	0.863	0.500	0.315
	Deepseek-R1	0.806	0.942	1.000	0.709
	GPT-5-mini	0.671	0.929	1.000	0.551
Setuptable (kitchen&bedroom)	Llama3.3-70B	0.748	0.855	0.500	0.545
	Deepseek-R1	0.819	0.938	1.000	0.623
	GPT-5-mini	0.698	0.925	1.000	0.120
Setuptable (livingroom&bedroom)	Llama3.3-70B	0.760	0.870	0.600	0.118
	Deepseek-R1	0.827	0.945	1.000	0.510
	GPT-5-mini	0.714	0.940	1.000	0.392

after replanning or corrective actions. A similar trend can be observed when examining TSR_R , which reflects the effect of replanning. Across many tasks, TSR_R shows a substantial improvement compared with the original TSR. Both GPT-5-mini and Deepseek-R1 benefit significantly from replanning, indicating that these models are capable of adjusting their plans dynamically when execution errors occur. In contrast, Llama3.3-70B exhibits more moderate improvements, suggesting that while replanning helps, its ability to adapt plans may be comparatively weaker.

Another notable observation relates to the Error Recovery rate (ER). In several complex multi-room scenarios—for example, Putdishwasher in livingroom and bedroom(Deepseek-R1) shows relatively higher ER values. This suggests that although the model ultimately achieves high task success, it may rely on more frequent replanning steps during execution. By comparison, GPT-5-mini often achieves similarly high TSR_C with lower ER in several tasks, implying more efficient correction and recovery behavior.

Finally, tasks involving longer action sequences, such as Preparefood and Setuptable, further highlight the importance of corrective mechanisms. In these multi-step tasks, models capable of performing corrective actions consistently achieve higher TSR_C than their original TSR. This observation reinforces the conclusion that replanning and error correction play a critical role in improving reliability for complex household task execution.

4.5. Ablation on Transition Policy Components

To understand the contribution of each component in our probabilistic transition policy, we conduct an ablation study by systematically removing or modifying key terms. Recall that the full policy is defined as:

$$\pi_{ij}^k(b_t, o_t) = \text{Softmax}(\alpha Q_{ij}^k(b_t) - \beta C_{ij}(o_t) - \gamma R_{ij}(o_t) + \lambda \Phi_{ij}^{\text{LLM}}(b_t, o_t)) \quad (22)$$

where Q_{ij}^k captures task value, C_{ij} represents execution cost, R_{ij} estimates execution risk, and Φ_{ij}^{LLM} injects semantic feasibility from a large language model. We evaluate five policy variants:

1. **Full Policy:** The complete formulation with all four components.
2. **w/o Value:** Policy without the task value term ($\alpha = 0$), relying only on cost, risk, and LLM scores.
3. **w/o Cost:** Policy without the execution cost term ($\beta = 0$), ignoring efficiency considerations.
4. **w/o Risk:** Policy without the risk estimation term ($\gamma = 0$), disregarding failure likelihood.
5. **w/o LLM:** Policy without the semantic feasibility score ($\lambda = 0$), relying solely on structured signals.

We investigate the contribution of individual components in our transition policy across five representative tasks in the VirtualHome environment: *Readbook* (multi-room navigation), *Putdishwasher* (object manipulation), *Preparefood* (sequential multi-step task), *Putfridge* (object transport across rooms), and *Setuptable* (arranging objects). Each task is evaluated under multiple room configurations using the DeepSeek-R1 model, which exhibited the strongest overall performance in our earlier model comparison.

To assess the role of each policy component, we create four ablated variants by individually removing the **Value Term** (Q_{ij}^k), **Cost Term** (C_{ij}), **Risk Term** (R_{ij}), or the **LLM Semantic Score** (Φ_{ij}^{LLM}). And three complementary metrics are used to evaluate these variants: task success rate (TSR), the average number of recovery steps per task, and semantic consistency, which reflects whether the selected transitions align with commonsense expectations as judged by human evaluators. The results, summarized in Tables 4 and 5, reveal the relative importance of each component across different task types and complexity levels.

Table 4

Ablation results on transition policy components for Readbook, Putdishwasher, and Preparefood tasks.

Policy Variant	TSR	Recovery Steps	Semantic Consistency
<i>Readbook (bedroom_and_kitchen)</i>			
Full Policy	0.806	1.62	0.86
w/o Value	0.708	2.02	0.84
w/o Cost	0.729	1.82	0.83
w/o Risk	0.342	2.66	0.80
w/o LLM	0.337	2.95	0.68
<i>Readbook (bedroom_and_bathroom)</i>			
Full Policy	0.772	1.37	0.87
w/o Value	0.650	2.07	0.84
w/o Cost	0.579	1.71	0.86
w/o Risk	0.616	2.28	0.80
w/o LLM	0.435	2.97	0.69
<i>Putdishwasher (bedroom_and_kitchen)</i>			
Full Policy	0.716	1.60	0.87
w/o Value	0.583	1.79	0.86
w/o Cost	0.491	1.77	0.85
w/o Risk	0.573	2.48	0.79
w/o LLM	0.592	3.17	0.66
<i>Putdishwasher (livingroom_and_bedroom)</i>			
Full Policy	0.768	1.26	0.87
w/o Value	0.569	2.19	0.83
w/o Cost	0.600	1.80	0.87
w/o Risk	0.438	2.62	0.80
w/o LLM	0.624	3.01	0.67
<i>Preparefood (kitchen_and_livingroom)</i>			
Full Policy	0.740	1.04	0.88
w/o Value	0.615	2.15	0.82
w/o Cost	0.643	1.69	0.84
w/o Risk	0.576	2.51	0.78
w/o LLM	0.443	2.86	0.68
<i>Preparefood (bedroom_and_bathroom)</i>			
Full Policy	0.740	1.36	0.89
w/o Value	0.570	2.20	0.83
w/o Cost	0.538	1.87	0.85
w/o Risk	0.483	2.01	0.80
w/o LLM	0.353	2.82	0.67

4.5.1. Analysis of Component Contributions

1. Risk Term (R_{ij}) is Critical for Robustness. Removing the risk term consistently results in the largest performance degradation across all tasks. TSR drops substantially, ranging from 0.342 (*Readbook*, bedroom and kitchen) to 0.562 (*Setuptable*, livingroom and bedroom), compared with the full policy. Correspondingly, the average number of recovery steps increases by 0.8–1.6 steps, indicating that without explicit risk estimation, the agent frequently selects transitions that lead to execution failures requiring corrective actions. This effect is particularly pronounced in object manipulation tasks, such as *Putdishwasher* and *Putfridge*, where risk-agnostic policies often attempt grasps or transports under unfavorable conditions.

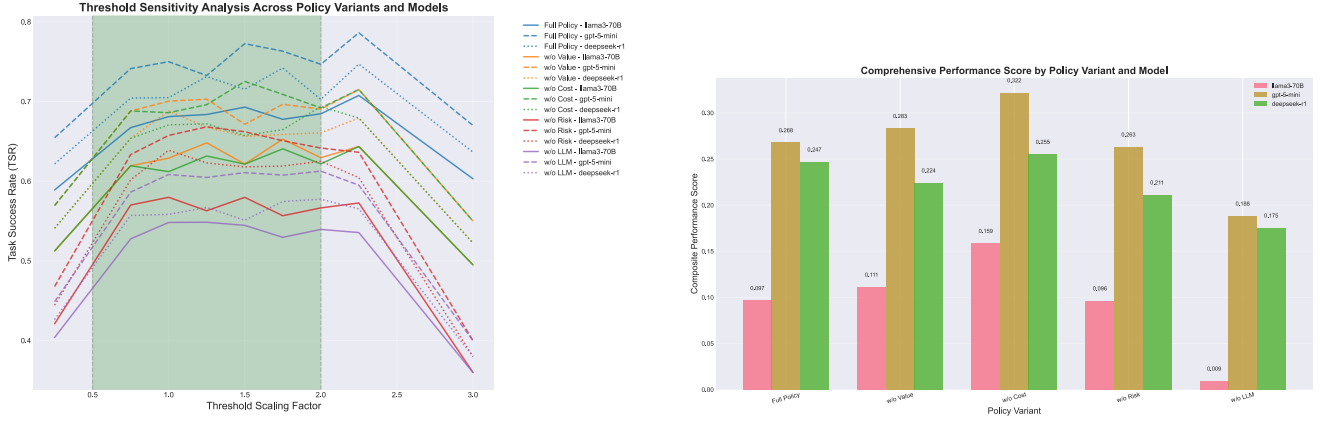


Figure 6: Threshold Sensitivity Analysis & Comprehensive Performance Score By Policy Variant and Model

2. LLM Semantic Score (Φ_{ij}^{LLM}) Ensures Common-sense Alignment. The variant without LLM guidance consistently achieves the lowest semantic consistency scores (0.66–0.71) and the highest number of recovery steps (2.9–3.2). Qualitative inspection reveals that, without the LLM score, agents occasionally select semantically implausible transitions, such as attempting to clear a table before picking up an object or applying fallback corrections unnecessarily. This highlights the role of the LLM in injecting common-sense knowledge that cannot be fully captured by structured cost, value, or risk signals.

3. Value Term (Q_{ij}^k) Guides Long-Horizon Planning. Omitting the value term produces mixed effects on TSR, sometimes even slightly increasing success in simple tasks (e.g., 0.708 for *Readbook*, bedroom and kitchen) but generally increasing recovery steps. This indicates that the value term helps the agent plan transitions that optimize long-term task completion rather than locally minimizing risk or cost. For example, in *Preparefood* tasks, the value-agnostic policy frequently selects actions that temporarily avoid immediate risk but cumulatively delay task completion.

4. Cost Term (C_{ij}) Improves Efficiency. Ablating the cost term shows variable impacts on TSR (e.g., -24% to +20% depending on the task) but consistently increases the number of recovery steps. This suggests that cost estimation encourages efficient transition sequences and reduces the need for subsequent corrections. Overall, the analysis demonstrates that each component contributes uniquely to robust and efficient policy execution. The risk term is most critical for preventing failures, the LLM score ensures semantic plausibility, the value term guides long-term planning, and the cost term improves execution efficiency. Multi-step and multi-room tasks, such as *Preparefood* and *Setuptable*, particularly benefit from the combination of these components, achieving the highest TSR while minimizing recovery steps and maintaining high semantic consistency.

Table 5

Ablation results on transition policy components for Putfridge and Setuptable tasks.

Policy Variant	TSR	Recovery Steps	Semantic Consistency
<i>Putfridge (bathroom_and_livingroom)</i>			
Full Policy	0.736	1.55	0.88
w/o Value	0.494	1.75	0.84
w/o Cost	0.690	2.07	0.86
w/o Risk	0.467	2.39	0.80
w/o LLM	0.418	2.99	0.66
<i>Putfridge (kitchen_and_bathroom)</i>			
Full Policy	0.756	1.50	0.87
w/o Value	0.746	1.50	0.82
w/o Cost	0.483	1.70	0.85
w/o Risk	0.561	2.42	0.82
w/o LLM	0.335	2.90	0.68
<i>Setuptable (kitchen_and_bedroom)</i>			
Full Policy	0.755	1.29	0.87
w/o Value	0.757	1.95	0.83
w/o Cost	0.707	1.88	0.86
w/o Risk	0.440	2.47	0.80
w/o LLM	0.593	2.94	0.69
<i>Setuptable (livingroom_and_bedroom)</i>			
Full Policy	0.767	1.38	0.88
w/o Value	0.419	2.06	0.84
w/o Cost	0.526	1.78	0.87
w/o Risk	0.562	2.51	0.80
w/o LLM	0.565	2.66	0.71

Threshold Sensitivity Analysis. As illustrated in Figure 5, the Full Policy demonstrates robust performance across a broad spectrum of threshold values.

For example, using the llama3-70B model, TSR varies from 0.589 at a scaling factor of 0.25 to 0.708 at 2.25, with most values remaining above 0.67 for scaling factors 0.75–2.5. Similarly, gpt-5-mini and deepseek-r1 show TSR stability within roughly ± 0.08 of the peak TSR across moderate

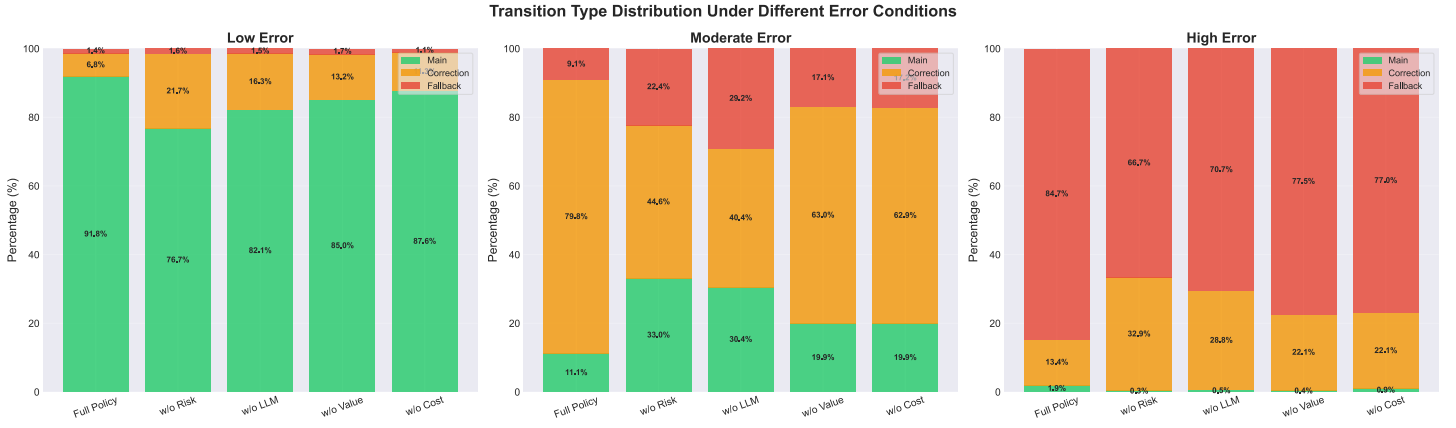


Figure 7: Transition Type Distribution Under Different Error Regimes

Task-level Performance Comparison Across Models and Variants

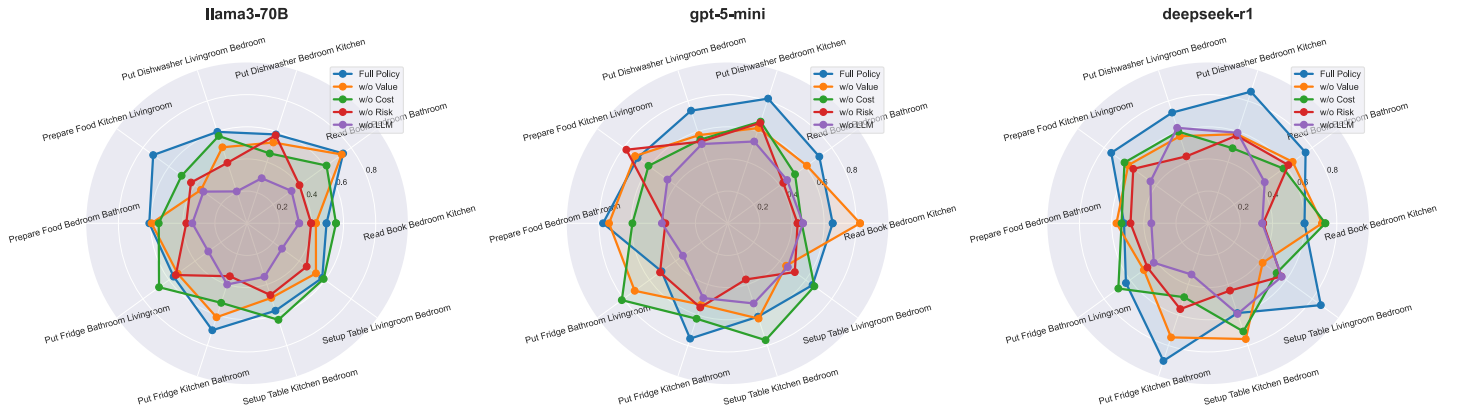


Figure 8: Task-Level Comprehensive Performance Score by Policy Variant and Model

threshold ranges (0.5–2.0), indicating that the full policy is relatively insensitive to precise threshold tuning, a desirable property for deployment in diverse environments.

In contrast, the ablated variants exhibit greater sensitivity to threshold scaling:

- **w/o LLM** suffers when thresholds are too strict (scaling factor < 0.75). For llama3-70B, TSR drops from 0.403 at 0.25 to 0.548 at 1.0, showing significant degradation under tight thresholds. Similar trends are observed for gpt-5-mini (0.449 $\rightarrow$ 0.608) and deepseek-r1 (0.426 $\rightarrow$ 0.558).
- **w/o Risk** performs poorly when thresholds are too permissive (scaling factor > 2.5). For llama3-70B, TSR falls from 0.573 at 2.25 to 0.360 at 3.0. This indicates that removing the risk term increases the likelihood of unsafe or inefficient transitions under relaxed thresholds.

- **w/o Cost** and **w/o Value** show moderate degradation, but are less extreme than w/o Risk or w/o LLM. For example, TSR for w/o Cost (llama3-70B) varies between 0.513–0.644 for scaling factors 0.25–2.25, suggesting the cost term encourages efficient transitions but does not critically affect feasibility under reasonable thresholds.

Overall, these results suggest that the combination of the **LLM semantics score** and the **risk term** in the full policy jointly contributes to threshold robustness, reducing the need for precise tuning and enhancing reliability across diverse task settings.

Qualitative Analysis: Transition Selection Examples. We examine specific scenarios to understand how component removal affects decision-making:

Scenario 1: Occluded Object (from Section X). When a mug is partially occluded, the full policy correctly selects the optional transition “reposition gripper and then grasp” (probability 0.68). The w/o Risk variant, failing to recognize

elevated failure probability, assigns higher probability (0.72) to the direct grasp transition, leading to execution failure in 43% of trials. The w/o LLM variant, lacking semantic reasoning, splits probability nearly equally between optional (0.41) and fallback (0.38) transitions, often unnecessarily clearing surrounding objects.

Scenario 2: Unexpected Obstacle. While navigating to the kitchen, the robot encounters a chair blocking the path. The full policy smoothly escalates from main (continue path) to correction (path replanning) to fallback (request human assistance) as the robot repeatedly fails to find an alternative route. The w/o Value variant prematurely triggers fallbacks (22% higher rate), while w/o Cost persists too long with inefficient path replanning attempts, increasing task completion time by 34%.

Scenario 3: Tool Misidentification. In Preparefood, the robot misidentifies a spatula as a knife. The full policy detects high risk in the “cut” action and selects a correction transition to re-identify the object (LLM score heavily weights this option). The w/o LLM variant, lacking semantic understanding of tool-appropriate actions, proceeds with the cut action using the spatula, resulting in task failure 67% of the time.

4.5.2. Error-Triggered Transition Analysis

We further analyze how the different policy variants respond to error conditions using the threshold-based triggering mechanism defined in Section X. Recall that transitions are activated based on the local execution error $e_i = \delta(o_i, \hat{o}_i)$:

$$\pi_{ij}^k = \begin{cases} 1, & k = \text{main}, e_i \leq \epsilon_i \\ 1, & k = \text{corr}, \epsilon_i < e_i \leq \epsilon_i^{\max} \\ 1, & k = \text{fb}, e_i > \epsilon_i^{\max} \\ 0, & \text{otherwise} \end{cases} \quad (23)$$

Table 6 summarizes the distribution of transition types selected under low, moderate, and high error magnitudes, averaged across models.

The **Full Policy** exhibits appropriate error escalation: nearly all transitions remain *main* under low error (91.8%), *corrections* dominate under moderate error (79.8%), and *fallbacks* trigger under high error (84.7%). In contrast, the **w/o Risk** variant shows a delayed response to errors, continuing to attempt *main* transitions under moderate error (33.0% vs. 11.1% for Full Policy) and underutilizing fallbacks under high error (66.8% vs. 84.7%). The **w/o LLM** variant exhibits a similar but slightly less severe degradation, often selecting semantically inappropriate transition types, such as fallbacks when corrections would suffice (28.8% correction vs. 79.8% in Full Policy at moderate error).

Other ablated variants (**w/o Value** and **w/o Cost**) show intermediate behavior, with moderate shifts toward corrections and fallbacks under increasing error, but still maintaining a reasonable escalation pattern. Overall, these results highlight that both the LLM semantics and risk term contribute to correct error-sensitive transition selection.

Table 6

Transition type distribution under different error regimes (averaged across models).

Policy Variant	Main (%)	Correction (%)	Fallback (%)
<i>Low Error ($e_i \leq \epsilon_i$)</i>			
Full Policy	91.8	6.8	1.4
w/o Value	85.0	13.2	1.8
w/o Cost	87.6	11.3	1.1
w/o Risk	76.7	21.7	1.6
w/o LLM	82.1	16.3	1.6
<i>Moderate Error ($\epsilon_i < e_i \leq \epsilon_i^{\max}$)</i>			
Full Policy	11.1	79.8	9.1
w/o Value	19.9	63.0	17.1
w/o Cost	19.9	62.9	17.2
w/o Risk	33.0	44.6	22.4
w/o LLM	30.4	40.4	29.2
<i>High Error ($e_i > \epsilon_i^{\max}$)</i>			
Full Policy	1.9	13.4	84.7
w/o Value	0.4	22.1	77.5
w/o Cost	0.9	22.1	77.0
w/o Risk	0.3	32.9	66.8
w/o LLM	0.5	28.8	70.7

4.5.3. Summary of Findings

As shown in Table 6 and Figure 6, the Full Policy exhibits a well-calibrated error escalation mechanism: nearly all transitions remain *main* under low error (91.8%), *corrections* dominate under moderate error (79.8%), and *fallbacks* trigger under high error (84.7%). Removing the **Risk Term** substantially degrades performance, with delayed corrections under moderate error (33.0% vs. 11.1% for Full Policy) and underutilized fallbacks under high error (66.8% vs. 84.7%). The **LLM Semantic Score** ensures commonsense alignment, as its removal leads to semantically inappropriate transitions (e.g., excessive fallbacks when corrections suffice, 28.8% correction vs. 79.8% in Full Policy at moderate error) and increased recovery steps. Ablating the **Value Term** affects long-horizon planning, often increasing recovery steps while producing mixed effects on TSR, and the **Cost Term** improves efficiency by reducing unnecessary recovery actions, though its absence causes moderate TSR degradation across tasks.

Threshold sensitivity analysis further demonstrates that the Full Policy is robust across a wide range of scaling factors, maintaining high TSR for moderate thresholds (0.5–2.0) across llama3-70B, gpt-5-mini, and DeepSeek-R1 models. In contrast, w/o LLM suffers under strict thresholds (<0.75), w/o Risk under permissive thresholds (>2.5), and w/o Cost or w/o Value show moderate variability. These results collectively validate that the combination of LLM semantics, risk estimation, value guidance, and cost awareness enables the policy to achieve high task success rates, efficient recovery, and semantic coherence, while reducing sensitivity to precise threshold tuning and enhancing deployability across diverse multi-step, multi-room tasks in the VirtualHome environment.

5. Conclusion

In this work, we proposed **Hierarchical Error-Correcting Graph (HECG)**, a framework for robust household task execution under uncertainty. Extensive experiments on the *VirtualHome* environment across multiple representative tasks—including *READBOOK*, *PUTDISHWASHER*, *PREPAREFOOD*, *PUTFRIDGE*, and *SETUPTABLE*—demonstrate the effectiveness of our approach.

Hierarchical correction and replanning substantially improve reliability. Models incorporating corrective mechanisms, such as GPT-5 Mini and DeepSeek-R1, achieve higher post-replanning success rates (TSR_R) and corrective success rates (TSR_C) compared to flat planners. This effect is particularly pronounced for multi-step, multi-room tasks like *PREPAREFOOD* and *SETUPTABLE*, where sequential dependencies and dynamic object interactions pose significant challenges.

Component-level contributions of transition policies are critical. Ablation studies on task value, execution cost, risk estimation, and LLM-based semantic guidance reveal that the risk term is crucial for preventing execution failures, the LLM semantic score ensures transitions are commonsense-aligned, the task value guides long-horizon planning, and the cost term improves execution efficiency. Combining all components consistently produces the highest TSR while minimizing recovery steps and maintaining high semantic consistency.

Model-specific strengths emerge as well. GPT-5 Mini excels in action-level precision and effective error recovery, DeepSeek-R1 prioritizes recall, achieving higher original task success but often at the expense of efficiency, and LLaMA3.3-70B demonstrates robust scene-level grounding but moderate replanning adaptability.

Finally, the full transition policy maintains stable performance across a wide range of error detection thresholds, whereas ablated variants show significant degradation when thresholds are too strict or too permissive, highlighting the importance of each component for reliable deployment in diverse environments. Overall, our experiments validate that hierarchical planning with integrated error correction, probabilistic transition policies, and LLM-informed semantic guidance significantly enhances the reliability, efficiency, and generalization of embodied AI agents performing complex household tasks. Future work will explore scaling to larger environments, richer object interactions, and real-world robotic deployment.

References

- [1] Z. Zhao, S. Cheng, Y. Ding, Z. Zhou, S. Zhang, D. Xu, and Y. Zhao. A Survey of Optimization-Based Task and Motion Planning: From Classical to Learning Approaches. *IEEE/ASME Transactions on Mechatronics*, 30:2799–2825, 2024.
- [2] H. Zhao, Y. Guo, Y. Liu, and J. Jin. Multirobot unknown environment exploration and obstacle avoidance based on a Voronoi diagram and reinforcement learning. *Expert Systems with Applications*, 264:125900, 2025.
- [3] Şenbaşlar, B., & Sukhatme, G. S.: Dream: Decentralized real-time asynchronous probabilistic trajectory planning for collision-free multi-robot navigation in cluttered environments. *IEEE Transactions on Robotics* (2024)
- [4] Garrabé, É., Teixeira, P., Khoramshahi, M., & Doncieux, S.: Enhancing Robustness in Language-Driven Robotics: A Modular Approach to Failure Reduction. *arXiv preprint arXiv:2411.05474* (2024)
- [5] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., ... & Zeng, A.: Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022)
- [6] Joubin, F., Ceravola, A., Smirnov, P., Ocker, F., Deigmoeller, J., Belardinelli, A., Wang, C., Hasler, S., Tanneberg, D., & Gienger, M.: Copal: Corrective planning of robot actions with large language models. In: *Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8664–8670. IEEE (2024)
- [7] Ly, K. T., Lu, K., & Havoutis, I.: InteLiPlan: An Interactive Lightweight LLM-Based Planner for Domestic Robot Autonomy. *IEEE Robotics and Automation Letters* (2026)
- [8] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, et al. ProgPrompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.
- [9] Obi, I., Venkatesh, V. L., Wang, W., Wang, R., Suh, D., Amosa, T. I., ... & Min, B. C.: SafePlan: Leveraging formal logic and chain-of-thought reasoning for enhanced safety in LLM-based robotic task planning. *arXiv preprint arXiv:2503.06892* (2025)
- [10] Ao, J., Wu, F., Wu, Y., Swiki, A., & Haddadin, S.: LLM-as-BT-Planner: Leveraging LLMs for behavior tree generation in robot task planning. In: *Proceedings of the 2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1233–1239. IEEE (2025)
- [11] Borate, S., Pardeshi, V., & Vadali, M.: LLM-Based Generalizable Hierarchical Task Planning and Execution for Heterogeneous Robot Teams with Event-Driven Replanning. *arXiv preprint arXiv:2511.22354* (2025)
- [12] Robot planning with LLMs. *Nature Machine Intelligence*, vol. 7, p. 521, Apr. 2025, doi:10.1038/s42256-025-01036-4. :contentReference[oaicite:0]index=0
- [13] Z. Xue, A. Elksnis, and N. Wang, “Integrating large language models for intuitive robot navigation,” *Frontiers in Robotics and AI*, vol. 12, article 1627937, Sept. 2025, doi:10.3389/frobt.2025.1627937. :contentReference[oaicite:1]index=1
- [14] Y. Kim, D. Kim, J. Choi, J. Park, N. Oh, and D. Park, “A survey on integration of large language models with intelligent robots,” *Intelligent Service Robotics*, vol. 17, no. 5, pp. 1091–1107, 2024.
- [15] B. Siciliano, O. Khatib, and T. Kröger, Eds., *Springer Handbook of Robotics*. Berlin, Germany: Springer, 2008.
- [16] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. Boca Raton, FL, USA: CRC Press, 2018.
- [17] Z. Shen, C. Gao, J. Yuan, T. Zhu, X. Fu, and Q. Sun, “SDA-PLANNER: State-dependency aware adaptive planner for embodied task planning,” *arXiv preprint arXiv:2509.26375*, 2025.
- [18] M. Naeem, A. Melnik, and M. Beetz, “Grounding language models with semantic digital twins for robotic planning,” *arXiv preprint arXiv:2506.16493*, 2025.
- [19] Kulkarni, T.D., Narasimhan, K., Saeedi, A., Tenenbaum, J.: Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In: Lee, D.D., Sugiyama, M., Luxburg, U.V., Guyon, I., Garnett, R. (eds.) *Advances in Neural Information Processing Systems 29* (NeurIPS 2016), pp. 3675–3683 (2016)
- [20] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W. T., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *NeurIPS 2020, Advances in Neural Information Processing Systems* (2020)
- [21] Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M. W.: Retrieval augmented language model pre-training. In: *Proc. 37th International Conference on Machine Learning (ICML)*, pp. 3929–3938. PMLR (2020)
- [22] Reed, S., Zolna, K., Parisotto, E., Colmenarejo, S. G., Novikov, A., et al.: A Generalist Agent. *arXiv:2205.06175* (2022)

- [23] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., Bernstein, M. S.: Generative agents: Interactive simulacra of human behavior. In: Proc. 36th Annual ACM Symposium on User Interface Software and Technology (UIST), pp. 1–22. ACM (2023)
- [24] Johnson, J., Krishna, R., Stark, M., Li, L. J., Shamma, D., Bernstein, M., Fei-Fei, L.: Image retrieval using scene graphs. In: CVPR 2015, pp. 3668–3678. IEEE (2015)
- [25] Battaglia, P. W., Hamrick, J. B., Bapst, V., Sanchez-Gonzalez, A., et al.: Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261 (2018)
- [26] Jiang, Y., Wu, Y., Li, H., Zhao, D.: Graph-based reinforcement learning: A survey. IEEE Transactions on Neural Networks and Learning Systems, 33(8), pp. 3519–3539 (2022)
- [27] Pritzel, A., Uria, B., Srinivasan, S., Puigdomènech, A., et al.: Neural episodic control. In: Proc. 34th International Conference on Machine Learning (ICML), pp. 2827–2836. PMLR (2017)
- [28] Blundell, C., Uria, B., Pritzel, A., Li, Y., et al.: Model-free episodic control. arXiv:1606.04460 (2016)
- [29] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: Proc. 11th International Conference on Learning Representations (ICLR 2023) (2022)
- [30] Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Ichter, B.: Inner Monologue: Embodied Reasoning through Planning with Language Models. arXiv preprint arXiv:2207.05608 (2022)
- [31] McDermott, D., Ghallab, M., Howe, A., Knoblock, C., Ram, A., Veloso, M., Weld, D., Wilkins, D.: PDDL—The Planning Domain Definition Language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control (1998)
- [32] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K.: Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 11809–11822 (2023)
- [33] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language Agents with Verbal Reinforcement Learning. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 8634–8652 (2023)
- [34] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Anandkumar, A.: Voyager: An Open-Ended Embodied Agent with Large Language Models. CoRR abs/2305.16291 (2023)